Horoscope and BaZi Memory Retrieval Agent

Horoscope and BaZi Memory Retrieval Agent

GitHub repository: https://github.com/ousyu66-pixel/information-retrieval
Video demo: https://drive.google.com/file/d/1sMczSJjGHewY6A9Zf-VqF-zYQ5gEJ4tR/view?usp=sharing

Introduction

This project implements the Horoscope and BaZi Memory Retrieval Agent, an AI agent that uses astrology as a creative domain for studying information retrieval in agent systems. The purpose of the system is not to make deterministic predictions, but to demonstrate how an agent can retrieve useful context before answering. Instead of directly generating a horoscope from a user prompt, the agent builds a context package from domain knowledge, persistent user memory, and date-specific temporal information.

System Design

The agent is implemented as a command-line Python application with an OpenAI-compatible chat completion interface. The system can run with `OPENAI_API_KEY`, `OPENAI_BASE_URL`, and `OPENAI_MODEL`, but it also has an offline preview mode that displays retrieved context without calling a model. This makes the retrieval behavior easier to inspect.

The agent has three main retrieval channels. First, it searches a local bilingual knowledge base containing Western astrology concepts such as zodiac signs, planets, houses, and moon phases, plus Chinese BaZi concepts such as 八字, 五行, 十天干, 十二地支, 日主, and 用神. The retriever uses a small BM25-style ranking algorithm implemented without external dependencies. Second, it retrieves personal context from a persistent memory file. This memory includes a user profile, prior questions and answers, and a compacted summary when the event log grows. Third, it retrieves temporal context from a structured JSON file of symbolic events for specific dates.

The agent follows a simple tool-use loop: read the user profile, build an enriched retrieval query, search the knowledge base, retrieve relevant memory, look up temporal context, synthesize the final answer, and save the forecast back to memory. Running the system with `--trace` shows these tool calls, making the agent architecture visible for evaluation.

Information Retrieval Contribution

The main IR contribution is the combination of different context sources before generation. A normal chatbot might respond from the prompt alone, while this agent explicitly retrieves domain knowledge, user memory, and time-specific data. The domain retrieval is intentionally multi-source: Western astrology and BaZi are treated as separate symbolic knowledge systems and fused only after retrieval. This improves explainability because the answer can include a “Retrieved context used” section. It also improves extensibility: future tools could add web search, document search over user journals, vector embeddings, or a skill router for different reflective practices. The BaZi extension strengthens the novelty of the project because it demonstrates cross-cultural context retrieval rather than a single-domain horoscope generator.

The system also includes a small compaction mechanism inspired by agent memory architectures. When the memory event list grows, older important events are compressed into a summary while recent events remain available for retrieval. This is a simplified version of long-context management used in more advanced agents.

Demo Interpretation

The demo video shows the agent running with a DeepSeek model through an OpenAI-compatible API. The user asks in Chinese: “My birthday is 2002-05-16, I want to focus on study, relationships, BaZi and five elements; please give me advice for today.” The generated answer is in Chinese, which shows that the model API was used for final synthesis rather than only the offline retrieval preview.

The trace demonstrates the agent workflow. First, `save_user_profile` extracts structured memory from natural language: `birth_date=2002-05-16`, `sun_sign=Taurus`, and `focus=study,relationship,bazi`. Then `get_user_profile` reads the stored profile back into the context. Next, `search_astro_knowledge` searches the local bilingual knowledge base and retrieves documents such as `Mercury Symbolism`, `Useful Element / 用神`, `Virgo / 处女座`, `Taurus / 金牛座`, `Seventh House: Partnership`, and `BaZi / 八字基础`. These results match the user’s topics: study, BaZi/five elements, personal sun sign, and relationships.

The tool `retrieve_user_memory` returns previous related runs from `memory/user_memory.json`, showing that the system has continuity across sessions. The tool `get_temporal_context` retrieves date-specific context for 2026-05-20, especially `Mercury review window`, which the answer uses to suggest reviewing messages, drafts, and assumptions. Finally, `save_forecast` stores the new answer back into memory. Therefore, the video output demonstrates an agent loop rather than a direct chatbot call: profile extraction, memory retrieval, domain retrieval, temporal retrieval, API synthesis, and memory update.

Reflection on Using AI Tools

I used AI coding tools to help build the overall framework of this project. During the process, I gained a clearer understanding of how an AI agent works in practice: it receives a user request, extracts useful state, calls tools, retrieves context, sends the organized context to a model, and saves useful information back to memory. At the same time, I am aware that there are still parts of the code-level implementation that I do not fully understand yet, especially the details of retrieval scoring, memory management, and API integration. This is something I need to continue studying carefully, because using AI to generate code is not enough; I also need to understand, check, and maintain the system myself.

References

OpenClaw documentation. Concepts including memory, tools, skills, heartbeat, and compaction.
Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond.
OpenAI API documentation. Chat completion style APIs and OpenAI-compatible model access.